Assignment 3: Budgeted Maximum Weight Clique using MPI

Deadline - 10th April 2026 EOD

Introduction

The goal of this assignment is to acquaint you with parallel programming with MPI. In this assignment, you will parallelize a variation of the Maximum Clique Problem.

You are given an undirected graph $G = (V, E)$. Each vertex $v \in V$ is associated with:

- A profit value $p(v) \in \mathbb{Z}^+$
- A cost value $c(v) \in \mathbb{Z}^+$

You are also given a maximum global budget B.

A subset of vertices $C \subseteq V$ is a *clique* if every two distinct vertices in C are adjacent. Your task is to find a clique C that maximizes the total profit:

$$P(C) = \sum_{v \in C} p(v)$$

Subject to the strict budget constraint:

$$\sum_{v \in C} c(v) \leq B$$

Your objective is to implement an exact sequential solver for this problem using a Branch and Bound approach, and then parallelize it using MPI to handle large, computationally intensive graphs.

Algorithm and Implementation Phases

1. Sequential Branch and Bound (Baseline)

Implement a recursive Branch and Bound algorithm to find the exact optimal clique.

- **Structural Bound (Greedy Graph Coloring):** Partition the candidate vertices into independent sets (color classes). Because a clique can contain at most one vertex from any independent set, the maximum structural profit is the sum of the highest single profit within each color class. To implement this greedily: sort the candidate vertices in descending order of their profit $p(v)$. Iterate through this sorted list, placing each vertex into the first available color class where it shares no edges with any existing members. If a vertex is adjacent to at least one member in every existing class, create a new color class for it.
- **Resource Bound (Fractional Knapsack):** Treat the remaining candidate vertices as items in a Fractional Knapsack problem. Sort them by their efficiency ratio $\frac{p(v)}{c(v)}$ in descending order. Greedily add the profits of these vertices allowing a fractional addition for the final vertex that exceeds the limit until the remaining budget is completely exhausted. This calculates the maximum theoretical profit assuming all remaining vertices could perfectly connect.

Algorithm 1 Sequential Branch and Bound for Budgeted Max Clique with Graph Coloring

```
Global: P_max ← 0                                   ▷ Stores the best profit found so far
procedure FindClique(C_cand, P_curr, W_curr)
    1. Structural Bound (Graph Coloring):
    Colors ← GreedyColor(C_cand)                    ▷ Partition into independent sets
    U_color ← Σ_{c∈Colors} max_{v∈c} p(v)           ▷ Max 1 vertex per color class
    if P_curr + U_color ≤ P_max then
        return                                      ▷ Prune: Graph structure prevents better profit
    end if
    2. Resource Bound (Fractional Knapsack):
    U_knap ← CalculateKnapsackBound(C_cand, B − W_curr)
    if P_curr + U_knap ≤ P_max then
        return                                      ▷ Prune: Budget constraint prevents better profit
    end if
    3. Branching:
    while C_cand is not empty do
        v ← Pop last element from C_cand
        if W_curr + c(v) ≤ B then
            P_max ← max(P_max, P_curr + p(v))
            C_next ← C_cand ∩ Neighbors(v)          ▷ Maintain clique property
            FindClique(C_next, P_curr + p(v), W_curr + c(v))
        end if
    end while
end procedure
```

2. MPI Parallelization

Parallelize the sequential solver using MPI. You have full creative freedom to design the parallelization strategy to achieve the best possible speedup.

- You must use the exact Branch and Bound algorithm and the two specific bounding heuristics described above. Tweaking, changing, or removing these bounds is not allowed.
- Use only MPI for parallelization. Hybrid approaches (like adding OpenMP, threads, or CUDA) are not permitted.

Input and Output Format

Input Rules

The input will be provided as a single text file

- **Line 1:** Three integers N (number of vertices), E (number of edges), and B (total budget).
- **Next N lines:** Each line contains two integers: $p(v)$ and $c(v)$ for vertex $v \in \{0, 1, ..., N-1\}$.
- **Next E lines:** Each line contains two integers u and v, representing an undirected edge between vertex u and vertex v.

Output Rules

Each file must contain exactly two lines:

- **Line 1:** The maximum profit found.
- **Line 2:** Space-separated vertex IDs that form the optimal clique, sorted in ascending order.

Example

Input:

```
4 5 10
4 3
5 4
6 5
7 5
0 1
0 2
1 2
1 3
2 3
```

Output:

```
13
2 3
```

Submission

- This assignment is to be done individually
- Submit a file named `a3_<your_entry_number>.zip`. For example, if your entry number is `2021CS12345`, your submission should be named `a3_2021CS12345.zip`.
  - Submit your C++ source files for the parallel Branch and Bound implementation. You may include any number of dependency files, but your submission must contain a `main.cpp` file that takes 2 arguments where the first is path of input graph and second is path of output file.
  - **Compilation and Execution:** We will compile and test your code on HPC using the following standard MPI commands. Make sure your submission compiles and runs perfectly with these exact commands:

    ```
    mpic++ -O3 -std=c++17 main.cpp -o main
    mpirun -np <num_processes> ./main input.txt output.txt
    ```

  - **Report:** Submit a brief PDF report named `report.pdf` that analyzes your results. This must include performance graphs (execution time vs. number of processes) showing how your code scales from 1 up to 16 ranks. Use the specific test graph that will be provided on Piazza for these measurements. Finally, include a short section explaining your parallelization strategy in your own words.

Evaluation

The total evaluation is out of 100 points, broken down as follows:

- **Correctness (30 points):** Your parallel implementation must yield the exact optimal clique and maximum profit. If your program produces the wrong answer, it will not be graded for performance.
- **Performance (70 points):** Points will be awarded based on the efficiency and scaling of your MPI implementation. The hidden test cases will utilize a combination of graphs with varying degrees of sparsity and density.